

A Sparse Polynomial Multiplier for HQC Integrating Parallelism and Power-Based Side-Channel Countermeasures

Jaeho Jeon¹, Suseong Lee¹, Myeongjun Kim², Eunyoung Seo¹, Myunghyun Cho³, Seonggyeom Kim³, Bo Gyeong Kang³ and Young-Sik Kim¹

¹ Department of Electrical Engineering and Computer Science, DGIST, Daegu, Korea, dgwogh@dgist.ac.kr, mercury@dgist.ac.kr, eyseo_dgist@dgist.ac.kr, ysk@dgist.ac.kr

² Department of Interdisciplinary Studies of Artificial Intelligence, DGIST, Daegu, Korea, sanmaru98u@dgist.ac.kr

³ Samsung Electronics System LSI Business, Hwaseong-si, Korea, mhyun.cho@samsung.com, sgyeom.kim@samsung.com, bogyong.kang@samsung.com

Abstract. The Hamming Quasi-Cyclic (HQC) scheme has recently been standardized as a post-quantum key encapsulation mechanism (KEM), emphasizing the importance of efficient and secure hardware realizations on embedded platforms. However, HQC relies heavily on sparse–dense polynomial multiplications, where conventional shift-and-add architectures remain both performance- and security-critical. In FPGA implementations, these multiplications dominate execution time—occupying 59.5%, 56.1%, and 58.3% of the total latency for KeyGen, Encap, and Decap, respectively—and are further vulnerable to correlation power analysis (CPA) due to deterministic, index-driven memory access patterns. As countermeasures, parallelization improves performance at the cost of additional area. Dummy insertion with random shuffling mitigates leakage but incurs extra cycle overhead.

To address this, we propose a co-designed **dummy-inserted parallel shift-and-add multiplier** for HQC. The design integrates dummy insertion and two-index parallelism in a complementary manner, achieving reduced cycles with area efficiency while providing intrinsic resistance to CPA. Implemented on a Xilinx Artix-7 FPGA, the proposed architecture achieves up to a **1.25× speedup** over the baseline sequential multiplier while maintaining near–state-of-the-art area–time efficiency—incurring only a **1.16× AT overhead** to simultaneously deliver accelerated performance and **CPA resistance**. Test Vector Leakage Assessment (TVLA) measurements and theoretical analysis confirm that the parallel architecture effectively suppresses power-based side-channel leakage and provides inherent resistance against CPA—reducing significant leakage points from 4.29% to 0.09%. This work demonstrates that performance and side-channel resistance can be jointly optimized through synergistic hardware–algorithm co-design, offering a practical and scalable HQC accelerator for post-quantum embedded systems.

Keywords: HQC · Hamming Quasi-Cyclic · BIKE · PQC · Post-Quantum Cryptography · KEM · FPGA · Embedded systems · side-channel attacks

1 Introduction

The advancement of quantum computing presents a significant threat to classical public-key cryptographic systems such as RSA [RSA78] and ECC [Mil86]. In particular, Shor’s algorithm [Sho94] has demonstrated the ability to efficiently break these schemes in polynomial time on a quantum computer. In response, the US National Institute of Standards and

Technology (NIST) initiated the Post-Quantum Cryptography (PQC) standardization process [BL17, CCJ⁺16]. As a result, in 2022, several PQC schemes, such as the lattice-based key encapsulation mechanism (KEM) CRYSTALS-Kyber [BDK⁺18], the lattice-based signature schemes CRYSTALS-Dilithium [DKL⁺18] and Falcon [FHK⁺18], and the stateless hash-based signature scheme SPHINCS+ [BHK⁺19], were selected as new NIST PQC standards. Recognizing that diversified assumptions about computational hardness are essential for robust cryptographic security, the standardization process emphasized the inclusion of schemes based on diverse mathematical foundations [AAC⁺22]. Accordingly, the fourth-round KEM finalists featured code-based candidates such as BIKE [ABB⁺22], Classic McEliece [BCL⁺17], and Hamming Quasi-Cyclic (HQC) [MAB⁺18], with HQC officially adopted as a new standard in 2025 [ABC⁺25].

Beyond theoretical robustness, NIST has underscored the importance of practical performance, particularly for embedded and resource-constrained hardware platforms [AAC⁺22]. This focus extends beyond general-purpose computing to include lightweight devices such as IoT nodes and smart cards, where efficient cryptographic implementations are vital for real-world deployment. Consequently, an increasing number of studies have explored FPGA- and ASIC-based implementations of PQC algorithms [DHSY24, XL21, ALCZ20].

In the case of HQC, several works have examined FPGA acceleration of its key components [DXN⁺23, THKX23, RBCGG22, ABP25, ABPS24], with particular emphasis on sparse–dense polynomial multiplication—a core and performance-critical operation. While the HQC specification and reference software deliberately avoid shift-and-add multipliers, whose control flow depends on secret sparse indices [MAB⁺18], many hardware designs still employ this approach for efficiency. Recently, a power side-channel key-recovery attack on a BIKE FPGA implementation [BZKG24]—which shares structural similarities with HQC—has demonstrated the practical vulnerability of such architectures, highlighting the need for more robust and side-channel-aware hardware countermeasures.

1.1 Motivation

Although code-based PQC schemes such as HQC provide strong theoretical security, existing hardware implementations face a critical trade-off: optimizing for **speed** compromises **security**, while hardening against **side-channels** degrades **performance**. Figure 1 shows this design conflict for HQC polynomial multiplication.

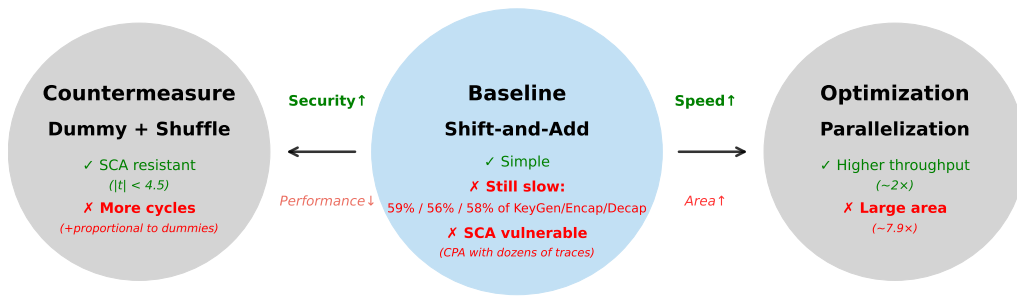


Figure 1: Performance–security trade-off in HQC polynomial multiplication. Dummy insertion improves side-channel resistance but increases latency, whereas parallelization reduces latency at the cost of area.

State-of-the-art (SOTA) FPGA implementations of HQC typically adopt the conventional shift-and-add architecture for sparse–dense polynomial multiplication, as it offers lower area and higher speed compared to Karatsuba or schoolbook methods.

- **Performance bottleneck.** The sequential nature of shift-and-add makes it inherently slow. In a full HQC hardware implementation by Deshpande et al. [DXN⁺23], polynomial multiplications dominate the total runtime—occupying 59.5% of Keygen, 56.1% of Encap, and 58.3% of Decap.
 - *Typical countermeasure:* Parallelization achieves higher throughput improvement.
 - *Trade-off:* Faster execution but significantly larger area overhead under single-port memory due to deep shift-register alignment.
- **Side-channel vulnerability.** The index-driven control flow leaks deterministic, cycle-aligned memory accesses that correlate with secret indices. Beckwith et al. [BZKG24] showed that such leakage enables full key recovery with only dozens of traces.
 - *Typical countermeasure:* Dummy insertion and shuffling achieve side-channel resistance.
 - *Trade-off:* Improved security but cycles proportional to dummy operations.

These two problems are fundamentally conflicting: randomization countermeasures slow execution, while throughput-oriented ones require much larger area. This conflict highlights the need for a new architecture that can simultaneously address both performance and side-channel resistance.

1.2 Contributions

Figure 2 maps existing HQC multiplier designs across area and latency. Individual countermeasures push toward opposite extremes: dummy insertion improves security but slows execution (①), while parallelization speeds up computation at significant area cost (②). Prior work thus trades performance for security, or vice versa.

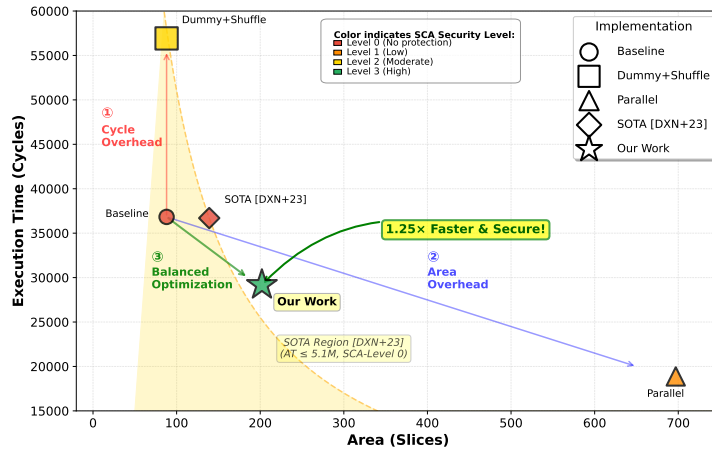


Figure 2: Hardware implementation trade-offs among HQC multiplier designs on Xilinx Artix-7 (xc7a200t, 32-bit bandwidth). The plot compares area (Slices) and latency (Cycles), with marker color indicating side-channel resistance level (0–3). The proposed co-designed architecture achieves **1.16× AT overhead compared to SOTA** while offering both **1.25× higher speed and strong side-channel resistance** compared with prior SOTA [DXN⁺23].

We **break this trade-off** by co-designing dummy insertion and parallelization so they reinforce each other. Inserting canceling dummy pairs bounds the maximum rotation distance, which in turn allows shallower shift registers for parallel execution. The result (arrow ③ in Fig. 2) is a multiplier that runs faster than the **baseline** while using minimal area overhead and providing strong side-channel protection.

This work makes three contributions:

1. **Co-designed dummy-parallel architecture.** We combine dummy insertion, randomized scheduling, and two-index parallelism to achieve both speed and security with low area overhead.
2. **Empirical validation.** Hardware measurements on a Xilinx Artix-7 (xc7a200t) demonstrate that the proposed design achieves a $1.25\times$ higher speed compared to the SOTA shift-and-add multiplier, while significantly improving first-order TVLA results (Fixed-vs-Random $|t| < 4.5$).
3. **Lightweight implementation.** The design maintains high efficiency under single-port 32-bit BRAM constraints, incurring only a $1.45\times$ area overhead to deliver enhanced performance and side-channel resistance, demonstrating its practicality for embedded and resource-constrained PQC hardware.

2 Preliminaries

2.1 Notation

Algebra and objects.

- $\mathbb{F}_2$: binary field. $\mathbb{F}_2[X]/(X^n - 1)$: ring of length- n binary polynomials modulo $X^n - 1$.
- Bold lowercase $\mathbf{x}, \mathbf{y}, \mathbf{r}_1, \mathbf{r}_2, \mathbf{e}$: length- n binary vectors (equivalently, polynomials in $\mathbb{F}_2[X]/(X^n - 1)$).
- $\oplus$: bitwise XOR (addition in $\mathbb{F}_2$). “ $\cdot$ ”: polynomial multiplication in $\mathbb{F}_2[X]/(X^n - 1)$.
- $\text{HW}(\cdot)$: Hamming weight (number of ones). $\text{HD}(a, b) = \text{HW}(a \oplus b)$: Hamming distance.
- $\text{Rot}_j(\mathbf{h})$: cyclic rotation of vector/polynomial $\mathbf{h} \cdot X^j \bmod (X^n - 1)$

Core HQC parameters.

- n : ring degree / vector length.
- ω : Hamming weight of secret sparse vectors $\mathbf{x}, \mathbf{y}$.
- ω_r ($= \omega_e$ in HQC spec): Hamming weight of ephemeral sparse vectors $\mathbf{r}_1, \mathbf{r}_2$ and noise $\mathbf{e}$.

Hardware-implementation symbols.

- n_d : number of inserted dummy *pairs*.
- **bandwidth**: memory interface width in bits (e.g., 32 bits in this work).
- **dense_BRAM**: BRAM storing the dense operand, divided into $W_{\text{dense}} = \lceil n / \text{bandwidth} \rceil$ words.

- **sparse_BRAM**: BRAM storing packed sparse indices. Each 32-bit word holds two 16-bit indices. In the **baseline (single)** mode, it stores ω words corresponding to all nonzero indices. In the **parallel** mode, dummy insertion increases the total count, resulting in about $\lceil(\omega + 2n_d)/2\rceil$ valid words.
- **acc_BRAM**: BRAM for the accumulator (size $\approx 2n$ bits), organized as $W_{\text{acc}} = \lceil 2n/\text{bandwidth} \rceil$ words.

2.2 HQC Specification

Recently, NIST has selected HQC KEM as a standard PQC KEM scheme [MAB⁺18]. HQC is a code-based public-key encryption (PKE) scheme whose IND-CPA security is proven based on the hardness of the syndrome decoding problem on random codes. It can be converted into a KEM that satisfies IND-CCA2 security. In this paper, we focus on the HQC-PKE procedures (Keygen, Encrypt, Decrypt), which form the hardware core, and refer to Appendix C for the KEM details. Table 1 summarizes the HQC core parameters used for the sparse polynomial multiplication considered in this paper. The algorithmic descriptions follow the official HQC specification [MAB⁺18].

Table 1: HQC instances and parameters relevant to sparse–dense multiplications (with $\omega_r = \omega_e$)

Instance	Security	n	ω	ω_r
HQC-1	NIST-1	17 669	66	75
HQC-3	NIST-3	35 851	100	114
HQC-5	NIST-5	57 637	131	149

HQC Keygen The key idea in the Keygen procedure lies in the sampling steps. First, `SampleFixedWeightVect` is executed with the set $\mathcal{R}_\omega$ to generate the vectors $\mathbf{x}$ and $\mathbf{y}$ of fixed Hamming weight ω . These vectors constitute the decryption key $\mathbf{dk}_{\text{PKE}}$ and represent the secret values of the scheme. Since ω is much smaller than the polynomial length n , both $\mathbf{x}$ and $\mathbf{y}$ are sparse. Next, the vector $\mathbf{h}$ is sampled uniformly at random from the entire space, and thus behaves as a dense polynomial. Finally, the dense vector $\mathbf{h}$ is multiplied with the sparse vector $\mathbf{y}$, and the result is combined with $\mathbf{x}$ to compute $\mathbf{s} = \mathbf{x} \oplus (\mathbf{h} \cdot \mathbf{y})$, which forms part of the encryption key $\mathbf{ek}_{\text{PKE}}$ (see Algorithm 1).

Algorithm 1 HQC-PKE.Keygen(seed_{PKE})

Require: Seed seed_{PKE}

Ensure: Encryption key $\mathbf{ek}_{\text{PKE}}$, decryption key $\mathbf{dk}_{\text{PKE}}$

- 1: (seed_{PKE.dk}, seed_{PKE.ek}) $\leftarrow$ I(seed_{PKE}) ▷ Compute $\mathbf{dk}_{\text{PKE}}$ and $\mathbf{ek}_{\text{PKE}}$ seeds
 - 2: ct_{PKE.dk} $\leftarrow$ XOF.Init(seed_{PKE.dk}) ▷ Compute decryption key $\mathbf{dk}_{\text{PKE}}$
 - 3: (ct_{PKE.dk}, $\mathbf{y}$) $\leftarrow$ SampleFixedWeightVect _{$\S$} (ct_{PKE.dk}, $\mathcal{R}_\omega$)
 - 4: (ct_{PKE.dk}, $\mathbf{x}$) $\leftarrow$ SampleFixedWeightVect _{$\S$} (ct_{PKE.dk}, $\mathcal{R}_\omega$)
 - 5: $\mathbf{dk}_{\text{PKE}} \leftarrow$ seed_{PKE.dk} ▷ Compute encryption key $\mathbf{ek}_{\text{PKE}}$
 - 6: ct_{PKE.ek} $\leftarrow$ XOF.Init(seed_{PKE.ek})
 - 7: (ct_{PKE.ek}, $\mathbf{h}$) $\leftarrow$ SampleVect(ct_{PKE.ek}, $\mathcal{R}$)
 - 8: $\mathbf{s} \leftarrow \mathbf{x} \oplus \mathbf{h} \cdot \mathbf{y}$ ▷ dense–sparse multiplication
 - 9: $\mathbf{ek}_{\text{PKE}} \leftarrow$ (seed_{PKE.ek}, $\mathbf{h}$, $\mathbf{s}$)
 - 10: **return** ($\mathbf{ek}_{\text{PKE}}$, $\mathbf{dk}_{\text{PKE}}$)
-

HQC Encrypt Based on the public encryption key ek_{PKE} produced in Keygen, the procedure encrypts the message $\mathbf{m}$. The encoder first reconstructs the public polynomial $\mathbf{h}$ (dense) from ek_{PKE} , and then samples fresh ephemeral randomness for this encryption: the fixed-weight vectors $\mathbf{r}_2$ and $\mathbf{r}_1$ (both sparse with weight ω_r), and the noise vector $\mathbf{e}$ (sparse with weight ω_e). The ciphertext components are computed as $\mathbf{u} = \mathbf{r}_1 \oplus (\mathbf{h} \cdot \mathbf{r}_2)$ and $\mathbf{v} = \text{C.Encode}(\mathbf{m}) \oplus \text{Truncate}(\mathbf{s} \cdot \mathbf{r}_2 \oplus \mathbf{e}, \ell)$, where $\mathbf{s}$ is the dense key part stored in ek_{PKE} . For each encryption, $\mathbf{r}_1$, $\mathbf{r}_2$, and $\mathbf{e}$ are re-sampled to guarantee IND-CPA security (fresh ciphertexts for the identical message) and to decorrelate cycle-level activity across traces, complementing dummy insertion and shuffling.

Algorithm 2 $\text{HQC-PKE.Encrypt}(\text{ek}_{\text{PKE}}, \mathbf{m}, \theta)$

Require: Encryption key ek_{PKE} , message $\mathbf{m}$, randomness θ

Ensure: Ciphertext $\mathbf{c}_{\text{PKE}}$

```

1: seedPKE,ek  $\leftarrow$  ekPKE[0:|seed|]
2: ctxPKE,ek  $\leftarrow$  XOF.Init(seedPKE,ek)
3: (ctxPKE,ek,  $\mathbf{h}$ )  $\leftarrow$  SampleVect(ctxPKE,ek,  $\mathcal{R}$ )
4:  $\mathbf{s} \leftarrow$  ekPKE[|seed|:|seed| + |s|]
5: ctx $_{\theta} \leftarrow$  XOF.Init( $\theta$ )
6: (ctx $_{\theta}$ ,  $\mathbf{r}_2$ )  $\leftarrow$  SampleFixedWeightVect(ctx $_{\theta}$ ,  $\mathcal{R}_{\omega_r}$ )
7: (ctx $_{\theta}$ ,  $\mathbf{e}$ )  $\leftarrow$  SampleFixedWeightVect(ctx $_{\theta}$ ,  $\mathcal{R}_{\omega_e}$ )
8: (ctx $_{\theta}$ ,  $\mathbf{r}_1$ )  $\leftarrow$  SampleFixedWeightVect(ctx $_{\theta}$ ,  $\mathcal{R}_{\omega_r}$ )
9:  $\mathbf{u} \leftarrow \mathbf{r}_1 \oplus \mathbf{h} \cdot \mathbf{r}_2$ 
10:  $\mathbf{v} \leftarrow \text{C.Encode}(\mathbf{m}) \oplus \text{Truncate}(\mathbf{s} \cdot \mathbf{r}_2 \oplus \mathbf{e}, \ell)$ 
11:  $\mathbf{c}_{\text{PKE}} \leftarrow (\mathbf{u}, \mathbf{v})$ 
12: return  $\mathbf{c}_{\text{PKE}}$ 

```

$\triangleright$ Parse encryption key ek_{PKE}
 $\triangleright$ Compute ciphertext $\mathbf{c}_{\text{PKE}}$
 $\triangleright$ dense-sparse multiplication
 $\triangleright$ dense-sparse multiplication

HQC Decrypt Given the decryption key dk_{PKE} and ciphertext $\mathbf{c}_{\text{PKE}} = (\mathbf{u}, \mathbf{v})$, the decoder regenerates the fixed-weight secret vector $\mathbf{y}$ and computes $\mathbf{m} = \text{C.Decode}(\mathbf{v} - \text{Truncate}(\mathbf{u} \cdot \mathbf{y}, \ell))$. The inner product $\mathbf{u} \cdot \mathbf{y}$ is a dense-sparse multiplication, as highlighted in Algorithm 3.

Algorithm 3 $\text{HQC-PKE.Decrypt}(\text{dk}_{\text{PKE}}, \mathbf{c}_{\text{PKE}})$

Require: Decryption key dk_{PKE} , ciphertext $\mathbf{c}_{\text{PKE}} = (\mathbf{u}, \mathbf{v})$

Ensure: Message $\mathbf{m}$

```

1: seedPKE,dk  $\leftarrow$  dkPKE[0:|seed|]
2: ctxPKE,dk  $\leftarrow$  XOF.Init(seedPKE,dk)
3: (ctxPKE,dk,  $\mathbf{y}$ )  $\leftarrow$  SampleFixedWeightVect(ctxPKE,dk,  $\mathcal{R}_{\omega}$ )
4:  $\mathbf{u} \leftarrow \mathbf{c}_{\text{PKE}}[0:|\mathbf{u}|]$ 
5:  $\mathbf{v} \leftarrow \mathbf{c}_{\text{PKE}}[|\mathbf{u}|:|\mathbf{u}| + |\mathbf{v}|]$ 
6:  $\mathbf{m} \leftarrow \text{C.Decode}(\mathbf{v} - \text{Truncate}(\mathbf{u} \cdot \mathbf{y}, \ell))$ 
7: return  $\mathbf{m}$ 

```

$\triangleright$ Parse decryption key dk_{PKE}
 $\triangleright$ Parse ciphertext $\mathbf{c}_{\text{PKE}}$
 $\triangleright$ Compute plaintext $\mathbf{m}$

Summary. Our main focus lies precisely on the parts highlighted in the above algorithms:

- In Algorithm 1, the computation $\mathbf{h} \cdot \mathbf{y}$ involves the dense public polynomial $\mathbf{h}$ and the sparse secret polynomial $\mathbf{y}$.

- In Algorithm 2, the computations $\mathbf{h} \cdot \mathbf{r}_2$ and $\mathbf{s} \cdot \mathbf{r}_2$ involve the sparse ephemeral secret vector $\mathbf{r}_2$ and the dense public-key components $\mathbf{h}$ and $\mathbf{s}$.
- In Algorithm 3, the computation $\mathbf{u} \cdot \mathbf{y}$ involves the dense ciphertext component $\mathbf{u}$ and the sparse secret polynomial $\mathbf{y}$.

These sparse–dense polynomial multiplications dominate the overall computation time and are known to be exploitable by power side-channel attacks. Accordingly, the following sections focus on optimizing and protecting these multiplications through hardware countermeasures such as dummy insertion and randomized scheduling.

2.3 Correlation Power Attack

Overview. Correlation Power Analysis (CPA) [BCO04] is a first-order side-channel attack that measures the linear correlation between predicted switching activity and measured power consumption. Given N traces of length L , the Pearson correlation for a key hypothesis κ is

$$\rho_j(\kappa) = \frac{\sum_{i=1}^N (t_j^{(i)} - \bar{t}_j)(h^{(i)}(\kappa) - \bar{h}(\kappa))}{\sqrt{\sum_{i=1}^N (t_j^{(i)} - \bar{t}_j)^2} \sqrt{\sum_{i=1}^N (h^{(i)}(\kappa) - \bar{h}(\kappa))^2}},$$

and the hypothesis yielding the highest $|\rho_j(\kappa)|$ is considered the correct key.

Leakage source. In CMOS circuits, dynamic power mainly arises from logic transitions; each 0→1 or 1→0 switch charges or discharges the load capacitance, making the number of bit toggles proportional to power consumption.

Leakage models. Two common models capture this behavior: **Hamming Weight (HW)** for data-dependent leakage, and **Hamming Distance (HD)** for transition-dependent leakage between consecutive states.

Attack process. The attacker defines a selection function $Z^{(i)}(\kappa) = f(\text{public data}^{(i)}, \kappa)$ for an intermediate value, computes the predicted leakage $h^{(i)}(\kappa) = \mathcal{L}(Z^{(i)}(\kappa))$ using HW or HD, and correlates it with measured traces. The key hypothesis producing the strongest correlation peak is selected.

2.4 Test Vector Leakage Assessment

We follow the Test Vector Leakage Assessment (TVLA) methodology proposed by Goodwill et al. [GGJR⁺11], which employs Welch’s two-sample t -test to statistically detect side-channel leakage. TVLA performs a pass/fail test to determine whether an implementation exhibits distinguishable data-dependent variations in its power traces. In essence, it evaluates whether two groups of traces—collected under different conditions—originate from the same underlying distribution. If the t -statistic remains small across all time samples, the implementation is considered non-leaking at the tested order.

Statistical model. Given two sets of traces, group 0 and group 1, with means μ_0, μ_1 , standard deviations σ_0, σ_1 , and sample sizes n_0, n_1 , the Welch’s t -statistic is computed as:

$$t = \frac{\mu_0 - \mu_1}{\sqrt{\frac{\sigma_0^2}{n_0} + \frac{\sigma_1^2}{n_1}}}.$$

This test does not assume equal variances and is thus well suited for real power measurements, which often exhibit heteroscedasticity due to noise or clock jitter. Large absolute values of $|t|$ indicate statistically significant differences between the two trace populations at the chosen significance level.

Fixed-vs-Random and Fixed-vs-Fixed tests. We employ two complementary TVLA variants:

- **Fixed-vs-Random :** One group uses a fixed secret key (or fixed sparse index set), while the other uses random keys per trace. The Fixed-vs-Random test checks whether the fixed-key traces can be statistically distinguished from random-key traces. A small $|t|$ across all samples implies the absence of first-order leakage—i.e., the fixed key appears random to an observer.
- **Fixed-vs-Fixed :** Both groups use different but fixed keys. This test highlights deterministic key-dependent structural differences even in the absence of randomness. Significant $|t|$ regions in Fixed-vs-Fixed reveal repeatable leakage patterns caused by cycle-aligned switching activity.

3 Previous Work

3.1 Shift-and-add Multiplier

As discussed in Algorithms 1, 2, and 3, the core products in HQC-PKE are $\mathbf{h} \cdot \mathbf{y}$, $\mathbf{h} \cdot \mathbf{r}_2$, $\mathbf{s} \cdot \mathbf{r}_2$, and $\mathbf{u} \cdot \mathbf{y}$. In each case, one operand is *dense* (public or ciphertext component) while the other is *sparse* and tied to secret or ephemeral secret material ($\mathbf{x}, \mathbf{y}, \mathbf{r}_2$). A naive schoolbook multiplication over $\mathbb{F}_2[X]/(X^n - 1)$ requires $O(n^2)$ bit-operations, producing all partial products (left of Fig. 3).

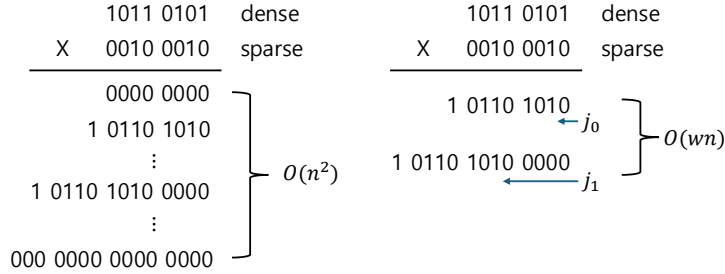


Figure 3: Comparison of schoolbook dense-sparse multiplication (left) versus shift-and-add (right). In the naive schoolbook approach every partial product is generated, resulting in $O(n^2)$ complexity. In the shift-and-add approach, only the ω nonzero positions of the sparse operand are processed, each producing a shifted copy of the dense operand ($j_0, j_1, \dots$), resulting in $O(\omega n)$ complexity.

As a result, many hardware implementations of code-based PQC schemes adopt the shift-and-add method for dense-sparse multiplications. For instance, Deshpande et al. [DXN⁺23] present a full HQC FPGA implementation based on a conventional shift-and-add multiplier, while Richter et al. [RBCGG22] report an FPGA implementation of BIKE, which employs the same sparse-dense multiplication principle but with different parameter sizes and code dimensions. Moreover, the LEAP multiplier [THKX23] focuses on HQC and provides a standalone optimized implementation of the multiplication unit.

Antognazza et al. [ABPS24] also implemented a complete HQC hardware architecture, maintaining the shift-and-add approach but exploring parallel configurations depending on the memory bandwidth and the number of memory ports, thereby improving throughput under various FPGA resource constraints.

Although the implementation details vary across these works, the underlying principle is the same: partial products of the dense polynomial are shifted according to the indices of the sparse operand and accumulated via XOR. In this work we developed our own baseline shift-and-add multiplier (Fig. 4) that captures this common logic, in order to provide a fair comparison with the proposed parallel architecture and to evaluate side-channel resistance under identical conditions.

Baseline configuration. All performance and security evaluations use the HQC-1 parameters, where $n = 17,669$ and $\omega = 66$, with a 32-bit memory interface (i.e., $\text{bandwidth} = 32$). The sparse indices are stored in `sparse_BRAM`, where each 32-bit word holds a single 16-bit index (one index per word for simplicity in the baseline). Since $\omega = 66$, this results in a total of **66** words stored in `sparse_BRAM`.

The dense operand is divided into $W_{\text{dense}} = \lceil n/\text{bandwidth} \rceil = \lceil 17,669/32 \rceil = 553$, and the accumulator polynomial, which has twice the length, requires $W_{\text{acc}} = \lceil 2n/\text{bandwidth} \rceil = \lceil 35,338/32 \rceil = 1,105$ words in `acc_BRAM`. Thus, `sparse_BRAM`, `dense_BRAM`, and `acc_BRAM` respectively store 66, 553, and 1,105 32-bit words in the HQC-1 baseline configuration.

When the multiplier receives a `start` signal, the controller issues a `sparse_addr` request to fetch the first index from `sparse_BRAM`. From the index value, it computes the target accumulator address (`acc_addr`) in word-level granularity, as well as the fine-grained bit shift. The word-level shift determines the offset into `acc_BRAM`, while the bit-level shift controls the alignment of the dense operand inside the `xor_adder` unit.

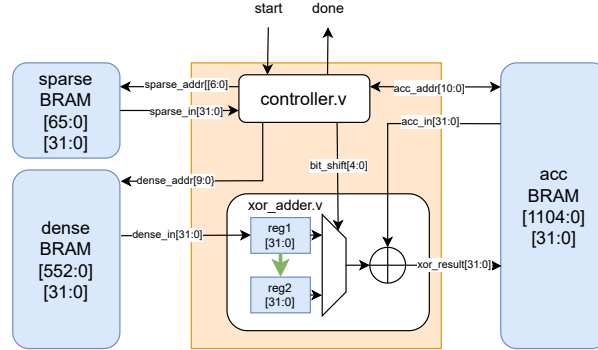


Figure 4: Block diagram of the baseline shift-and-add multiplier. The design consists of three main BRAMs (sparse, dense, and accumulator), a controller, and the `xor_adder` module that performs word- and bit-shifts followed by XOR accumulation.

Example. For instance, suppose that the first index fetched from `sparse_BRAM[0]` is 148. Since the memory bandwidth is 32 bits, the corresponding word and bit offsets are computed as

$$\text{word_shift} = \left\lfloor \frac{148}{32} \right\rfloor = 4, \quad \text{bit_shift} = 148 \bmod 32 = 20.$$

In this case, the controller requests `acc_BRAM[4]` for accumulation and forwards `bit_shift = 20` to the `xor_adder.v` module, which performs a 20-bit left rotation of the dense input word before the XOR operation. This example illustrates how the system separates the

coarse (word-level) address computation from the fine-grained (bit-level) alignment within the datapath.

The `xor_adder.v` module contains two registers that can delay and align the dense input by the required bit shift. Each 32-bit word from `dense_BRAM` is shifted accordingly, then XORed with the corresponding `acc_in` value from `acc_BRAM`. The result (`xor_result`) is written back into `acc_BRAM`. This process is repeated for all 553 dense words.

The operation continues iteratively for all $\omega = 66$ nonzero positions of the sparse operand. Consequently, the total latency of the baseline design can be expressed as:

$$\text{cycles}_{\text{baseline}} = \omega \cdot (W_{\text{dense}} + 5) + 2,$$

where W_{dense} is 553. For the HQC-1 configuration, $\text{cycles}_{\text{baseline}}$ is evaluated as approximately 36,830 cycles.

3.2 CPA on shift-and-add Multiplier

Beckwith et al. [BZKG24] present a practical Correlation Power Analysis (CPA) key-recovery attack against an FPGA implementation of BIKE that uses a sequential shift-and-add multiplier (the RacingBIKE design [RBCGG22]). The attack demonstrates that the sequential, index-driven nature of the multiplier produces a large number of exploitable intermediate values within each power trace, enabling coefficient-by-coefficient recovery of the sparse secret polynomial with relatively few traces.

In Fig. 5 we indicate two classes of datapath signals relevant to the attack: the *red-dashed* region (the signals actually exploited in Beckwith et al.) and a *yellow-dashed* region (a plausible secondary leakage point). Concretely, the red-dashed signals correspond to the accumulator I/O in our baseline multiplier (see Fig. 4), i.e. the word-level inputs/outputs that appear as `acc_in[31:0]` (these are the signals labeled as `resa_in`, `resb_in` in the reference design). The yellow-dashed region corresponds to the final write-back path (the value we call `xor_result[31:0]`, labeled as `res_out` in the reference figure). Both sets of signals show secret-dependent, cycle-aligned activity in a sequential shift-and-add datapath.

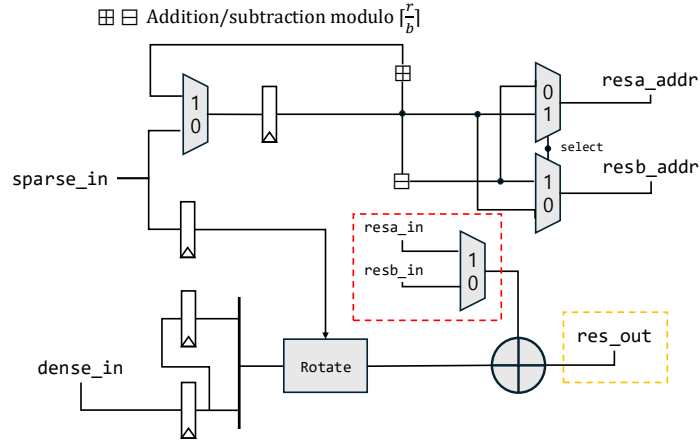


Figure 5: Target datapath of the CPA attack in BIKE’s shift-and-add multiplier, adapted from Beckwith et al. [BZKG24]. The leakage is concentrated around accumulator memory I/O signals (`resa_in`, `resb_in`), which CPA exploits to recover sparse secret coefficients. The write-back signal (`res_out`) is also a plausible leakage point.

The attack targets the decapsulation step, in which a dense ciphertext polynomial is multiplied by a secret sparse polynomial h_0 . In BIKE-1 the relevant parameters are

$r = 12,323$ and $\omega = 71$. The attacker forms hypotheses for the nonzero index/rotation of each coefficient of h_0 (i.e., guesses in the range $0 \dots r - 1$), and for each hypothesis computes the corresponding intermediate memory outputs of the shift-and-add datapath. The primary leakage target in the implementation is the accumulator memory I/O (signals denoted `resa_in`, `resb_in`), which correlates strongly with measured power samples.

Operationally, Beckwith et al. divide the recovery into three phases:

1. Recover the first coefficient $h_0[0]$ by exploiting how its induced shifts affect the rotated words written to the accumulator.
2. Recover the word-shift bits of subsequent coefficients using previously recovered values to reduce the hypothesis space.
3. Recover the remaining bit-shift bits by modeling memory read/write behavior and correlating predictions with power traces.

For each guess, the adversary computes hypothetical memory outputs (using public c_0 and partial key information) and applies a Hamming-distance leakage model against measured traces to select the most likely hypothesis.

Beckwith et al. report very low Measurement-to-Disclosure (MTD) figures: full key recovery at BIKE-1 security level typically requires only a few dozen traces (average MTD ≈ 17 traces). The attack needs only known ciphertexts and power traces — no chosen-ciphertext or profiling is required.

Illustrative example (baseline). To clarify the leakage model of the baseline shift-and-add datapath, consider two consecutive sparse indices stored in `sparse_BRAM`: `sparse_BRAM[0] = 148` and `sparse_BRAM[1] = 342`. When processing index 148, the shifted copy of the dense operand is XORed into the accumulator at the corresponding word address, producing a concrete 32-bit value that is later re-read from `acc_BRAM`. When index 342 is serviced, the datapath fetches `acc_in` from the same accumulator region, now containing the earlier contribution from 148.

An attacker, knowing the dense public operand, can hypothesize secret indices $\kappa \in [0, n-1]$ (here $n=17,669$). For each guess, the attacker computes the expected accumulator value on the `acc_in` bus—denoted $Z = f(\text{public dense}, \kappa)$ —using the known dense data and the guessed sparse rotation. A Hamming-distance or Hamming-weight model $\mathcal{L}(Z)$ yields the predicted leakage $h^{(i)}(\kappa)$, which is then correlated with measured power traces around the corresponding cycle. The hypothesis showing the highest correlation is taken as the correct index. In the sequential baseline design, this per-cycle determinism allows coefficient-by-coefficient recovery of the secret polynomial when traces are aligned.

Implementation-independence of the leakage. It is important to emphasise that the red/yellow leakage points identified above are not implementation-specific quirks of a single FPGA design. Rather, they stem from the fundamental control flow of any sequential, index-driven shift-and-add multiplier (“read sparse index $\rightarrow$ rotate dense operand $\rightarrow$ XOR into accumulator”). Consequently, regardless of implementation choices such as word width, BRAM arrangement, or minor pipelining, the same per-index, cycle-aligned memory events occur and can produce secret-dependent intermediate values that CPA can exploit. This implementation-independence motivates our focus on architectural changes (parallel datapaths) and lightweight countermeasures (dummy insertion and randomized scheduling) in Section 4.

Summary of previous work. The attack model and leakage characteristics above underscore the need for a redesigned multiplication architecture that simultaneously improves throughput and mitigates power leakage.

4 Proposed Method

The vulnerabilities and performance bottlenecks identified in the previous section motivate a new hardware architecture for HQC that balances efficiency and security. Our design objectives are twofold:

- **Increase parallelism and throughput** compared to the sequential shift-and-add baseline.
- **Reduce data-dependent switching activity and side-channel leakage**, thus improving resistance to power-based attacks such as CPA.

To achieve these goals, we propose a parallel sparse–dense polynomial multiplier architecture tailored for HQC, incorporating three lightweight countermeasures: (i) dummy insertion, (ii) random shuffling, and (iii) shift-register-based alignment. Together, these techniques randomize the order and location of secret-dependent operations while maintaining compact area and minimal control overhead. The following subsections describe the concrete implementation of the proposed countermeasure.

4.1 Dummy Insertion

Dummy insertion is a lightweight countermeasure that simultaneously hides power leakage and enables parallel execution. By duplicating selected sparse indices, we inject extra switching activity that obscures which memory accesses correspond to real key material.

Rationale for correctness. In the HQC multiplier, all arithmetic operations are performed on $\mathbb{F}_2[X]/(X^n - 1)$, where addition corresponds to bitwise XOR. Therefore, inserting dummy operations—i.e., redundant multiplications with repeated sparse indices—does not alter the final result. Each multiplication in the shift-and-add process performs an XOR accumulation between the shifted dense operand and the accumulator. If the same index is processed twice, its contribution is XORed twice:

$$\mathbf{a} \oplus \text{Rot}_{j_t}(\mathbf{b}) \oplus \text{Rot}_{j_t}(\mathbf{b}) = \mathbf{a},$$

which cancels out in $\mathbb{F}_2$.

Illustrative example (HQC-1). To clarify the dummy-insertion process, we illustrate it using the HQC-1 parameter set ($n=17,669$, $\omega=66$) with 18 inserted dummies. Initially, 66 valid sparse indices $\{j_0, j_1, \dots, j_{65}\}$ are stored in ascending order, as shown on the left of Fig. 6. Dummies $\{d_0, d_1, \dots, d_{17}\}$ are then inserted at positions where the gaps $\Delta_i = j_{i+1} - j_i$ exceed a predefined threshold (e.g., 512). Each insertion effectively splits an original pair $\delta_i = (j_i, j_{i+1})$ into two shorter pairs, reducing the local spacing between valid indices.

For example, consider $\delta_0 = (j_0, j_1)$. When a dummy d_0 is inserted between them, the pair is split into $\delta_0^* = (j_0, d_0)$ and $\delta_{33}^* = (d_0, j_1)$, as depicted in Fig. 6 (right). After inserting all 18 dummies, the total number of 32-bit words increases to 51, each containing two indices. Consequently, large gaps are replaced by smaller, bounded intervals, enabling predictable index spacing for subsequent parallel processing.

Enabling parallelism. Without dummy insertion, the maximum inter-index distance varies widely and is hard to predict, forcing the design to provision shift registers up to 4096 bits to accommodate worst-case gaps. This variability prevents a compact, fixed-depth parallel aligner. By bounding all adjacent distances to ≤ 512 through deterministic dummy insertion, we obtain a predictable maximum delay that maps directly to a 17-tap

shift-register array (Section 4.2), enabling efficient, fixed-depth parallel processing with consistent timing.

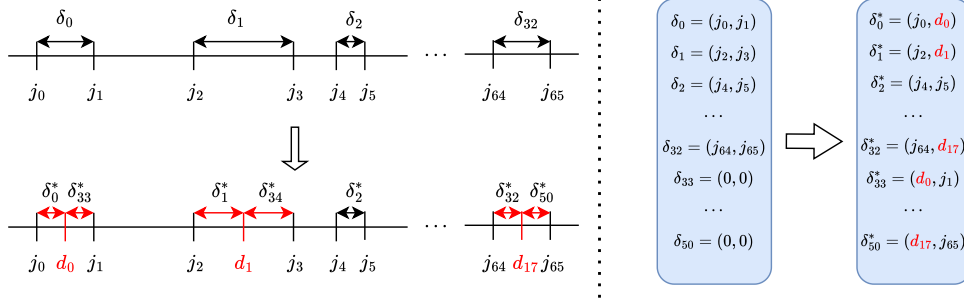


Figure 6: Illustration of the dummy insertion process. The left side shows the original index distribution and corresponding gap values δ . Dummies are inserted at positions where $\delta > 512$, and the right side depicts how empty slots in `sparse_BRAM` are filled with duplicated pairs (j, j) as insertion proceeds.

Hardware architecture of dummy insertion. The dummy-insertion module operates as a simple, fixed-time controller that scans all index pairs and inserts additional dummy values to ensure that every pair of consecutive indices lies within a distance of 512 or less. It iteratively applies four thresholds $T_k = \{4096, 2048, 1024, 512\}$ in descending order, as summarized in **Algorithm 4** (Appendix B). For each threshold pass, all index pairs (w_{2i}, w_{2i+1}) are examined: if their distance $\delta = w_{2i+1} - w_{2i}$ exceeds T_k , a new dummy index $w_{2i} + T_k$ is inserted between them, reducing the gap. After the final pass ($T_3 = 512$), any remaining dummies are evenly distributed by splitting the residual gaps (i.e., halving distances already below 512).

This deterministic procedure guarantees that all adjacent indices are spaced within 512 positions, enabling efficient mapping to the 17-tap shift-register structure in the parallel multiplier. The entire process completes in a constant **212 cycles**, independent of the input index distribution, providing both timing consistency and minimal control complexity.

Once generated during **Keygen**, the dummy-augmented sparse indices can be reused across all subsequent decapsulation (**Decap**) operations for the same key without regeneration. In contrast, during **Encap**, a new ephemeral secret vector r_2 is sampled for every encryption; hence the dummy-insertion module incurs an additional **230-cycle** cost per instantiation to populate its dummy pairs.

Design flexibility. This approach is presented as a representative implementation—users may instead introduce randomness into the dummy placement, precompute dummies in software, or perform the process offline. Moreover, the number of inserted dummies can be tuned to trade off latency and area: increasing n_d shortens index gaps (reducing shift-register depth) at the cost of additional multiplication cycles. In this work, we adopt a fixed-threshold, deterministic dummy-insertion scheme to focus on optimizing area and cycle efficiency within the hardware design.

Table 2: Area and performance results of the **dummy insertion module** on Xilinx Artix-7 xc7a200t FPGA. Left value corresponds to ω (secret weight) and right value to ω_r (ephemeral weight).

Param Set	n_d	FPGA Utilization				Latency (cycles)
		Slices	LUT ($\times 10^3$)	FF ($\times 10^3$)	BRAM	
HQC-1	(18, 14)	42	240	38	1	(212, 230)
HQC-3	(36, 34)	47	278	39	1	(346, 378)
HQC-5	(58, 54)	48	301	38	1	(534, 572)

4.2 Parallel Multiplier

Overview. After dummy insertion, the multiplier operates on a 50-word **sparse_BRAM** (each 32-bit word packs two 16-bit indices) together with **dense_BRAM**. A permutation of the 50 words is applied so that two indices (packed in one word) are processed in parallel via a shift-register-based datapath, while the dense stream remains deterministic.

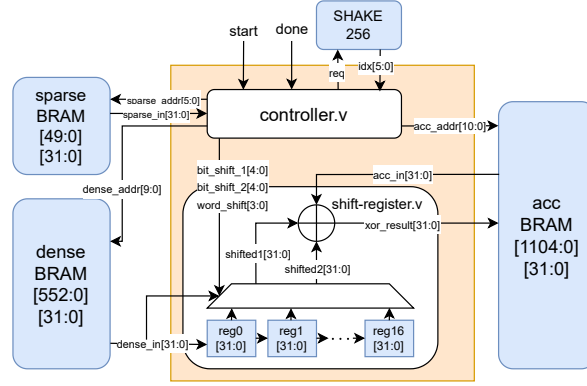


Figure 7: Shuffle + shift-register datapath overview. A SHAKE256-driven shuffle supplies randomized $\text{idx} \in [0, 49]$ to fetch **sparse_in** (two packed indices). The controller derives per-index shifts **word_shift_1**, **bit_shift_1** and **word_shift_2**, **bit_shift_2**; the **shift-register** aligns the two dense contributions in parallel before XOR accumulation into **acc_BRAM** at the corresponding word addresses.

Shuffle via SHAKE256. The controller derives pseudorandom indices from the HQC-internal SHAKE256 stream and enforces a permutation using a 50-bit *visited bitmap*. It draws 16-bit candidates from SHAKE256, reduces each modulo 50 to obtain $\text{idx} \in [0, 49]$, and accepts the first candidate whose bitmap bit is clear (otherwise, it discards the candidate and continues scanning). *Crucially, this candidate generation runs in parallel with the datapath:* while the current index word is being processed, the SHAKE256 engine streams candidates into a small FIFO and the bitmap filter resolves the next unused idx in the background. By the time the datapath finishes the current iteration, a ready idx is already available, so $\text{sparse_addr} \leftarrow \text{idx}$ can be issued without wait states. This overlap hides any internal rejections, preventing rejection-induced timing variation on the critical path and preserving identical functionality while degrading trace-to-trace cycle alignment.

Randomized fetch and shift computation. For each accepted `idx`, the controller requests `sparse_addr[5:0]` and receives `sparse_in[31:0]`, i.e., two packed indices (j_{lo}, j_{hi}). The word-level offset between these indices is computed directly from their 32-bit word indices as

$$\text{word_shift} = \left\lfloor \frac{j_{hi}}{32} \right\rfloor - \left\lfloor \frac{j_{lo}}{32} \right\rfloor.$$

The intra-word alignments are taken per index as

$$\text{bit_shift_1} = j_{lo} \bmod 32, \quad \text{bit_shift_2} = j_{hi} \bmod 32.$$

These fields drive the `shift-register` so that both indices in a word are serviced in parallel.

Parallel alignment via shift registers. The `shift-register.v` keeps a 17-tap register array (`reg0-reg16`) buffering `dense_in[31:0]` by programmable word delays (0–16). A small barrel shifter applies `bit_shift_k` per index. Because dummy insertion bounds inter-index gaps to ≤ 512 bits (16 words), 17 taps suffice to align both contributions. The aligned words `shifted1` and `shifted2` are XORed with `acc_in[31:0]` to produce `xor_result[31:0]`, written back to `acc_BRAM` at `acc_addr[10:0]`. The `dense` stream is deterministic; only tap selection varies with the SHAKE256-driven order. To minimize resources, the delay line is implemented with Xilinx’s LUT-based shift-register primitive `SRLC32E`¹.

Cycle estimation. The total latency is

$$\text{cycles}_{\text{parallel}} = (W_{\text{dense}} + N_{\text{shift_reg}} + 2) \times \left\lceil \frac{\omega + 2n_d}{2} \right\rceil + 1,$$

where all terms inside the first parentheses are *constants* for a given configuration. In our HQC-128 setup, $W_{\text{dense}} = \lceil n/\text{bandwidth} \rceil = 553$, and $N_{\text{shift_reg}}$ follows directly from the maximum allowed inter-index distance of 512 bits: $N_{\text{shift_reg}} = \lceil 512/32 \rceil + 1 = 17$. With $\omega = 66$ and $n_d = 17$, this evaluates to $\approx 28,751$ cycles.

Table 3: Area and performance results of the `shift-register module` on Xilinx Artix-7 xc7a200t FPGA.

Param	FPGA Utilization				n_d [†]	Latency (cycles)
	Slices	LUT	FF	BRAM (36Kb)		
HQC-1	202	657	159	2	18 / 14	29,326 / 29,745
HQC-3	200	648	164	3	36 / 34	98,041 / 103,741
HQC-5	209	648	164	5	58 / 54	225,805 / 234,910

Parameter summary. For HQC- $\{1,3,5\}$, the secret weights are $\omega = \{66, 100, 131\}$ (“left”) and the ephemeral/noise weights are $\omega_r = \{75, 114, 149\}$ (“right”). The ring sizes are $n = \{17,669, 35,851, 57,637\}$, respectively. [†] n_d is the number of inserted *dummy pairs*. The entry “right/left” lists the counts used for ω_r (right) and ω (left), e.g., **18 / 14** means $n_d=18$ for ω_r and 17 for ω at HQC-1.

¹<https://docs.amd.com/r/en-US/ug953-vivado-7series-libraries/SRLC32E>. An addressable 32-bit shift register with clock enable and tap selection ($A[4:0]$), providing low-LUT/FF usage and easy cascading for deeper delays (outputs Q , $Q31$).

5 Result

5.1 Side-Channel Security

Experimental setup. We captured power traces on a *Chip Whisperer* CW305 target (Xilinx Artix-7 XC7A35T) [OC14], clocked at 2.5 MHz, using a PicoScope 3403D at 500 MS/s (≈ 200 samples per device clock cycle), with hardware triggering the multiplier start signal.

Claim 1 — Parallel + Dummy + Random Co-Design Increases CPA Complexity. We evaluate how the proposed parallel multiplier with dummy insertion and SHAKE256-driven shuffling affects standard leakage tests and the CPA attack model of Beckwith et al. [BZKG24].

1.1 Increased CPA hypothesis complexity (from $\Theta(n)$ to $\Theta(n^2)$ per iteration). Under the sequential shift-and-add model, an attacker formulates a single-index hypothesis for each sparse access, e.g., $h(0), h(1), \dots, h(n-1)$ for indices in the range $[0, n-1]$, so the search space is $\Theta(n)$. In our parallel scheduler, two sparse indices contribute simultaneously in the same iteration, so a CPA must consider *pairs* of the indices rather than singletons. Concretely, the attacker now needs to enumerate hypotheses of the form $h(i, j)$ (e.g., $h(0, 1), h(0, 2), \dots$), rather than $h(i)$ alone. This expands the effective hypothesis space from n to $\Theta(n^2)$, increasing the correlation-search time and creating greater ambiguity among competing hypotheses, thus raising the practical work factor for CPA even with large trace counts.

1.2 Parallel alignment hides the cycle-level power pattern (Fixed-vs-Fixed TVLA). We ran Fixed-vs-Fixed TVLA on two distinct keys for the baseline single-path and our parallel path. This test highlights a repeatable key-dependent structure even without randomness. To isolate the intrinsic effect of parallelism, this experiment excludes dummy insertion and random shuffling—evaluating the parallel datapath alone. As shown in Fig. 8, the single-path design exhibits a higher fraction of significant points ($|t| > 4.5$) at **1.62%**, whereas the parallel design reduces this to **1.26%**. Intuitively, with two secret-controlled contributions aligned per iteration (and taps determined by `word_shift/bit_shift`), the deterministic per-cycle pattern of the baseline is blurred, increasing apparent noise, and suppressing the consistent alignment that CPA exploits across traces.

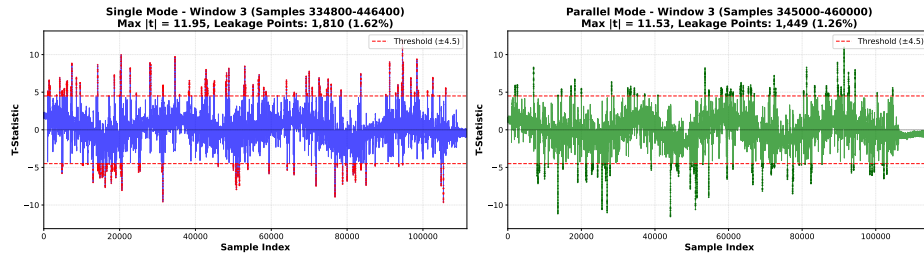


Figure 8: Fixed-vs-Fixed TVLA comparison around the processing of `sparse_BRAM[3]` (2000 traces). All other sparse indices $\{j_0, \dots, j_{65}\}$ are identical across both configurations, except for `sparse_BRAM[3]`, which is intentionally varied to isolate its effect. The upper plot shows the baseline *single-path* mode (**1.62%** significant points), and the lower plot shows the proposed *parallel* mode (**1.26%** significant points). Parallel execution visibly reduces the density of significant t -values, indicating weaker cycle-aligned correlation localized to this BRAM access.

1.3 Dummy insertion + shuffling obscures which hits correspond to real key indices.

Even if an attacker observes residual peaks, dummy insertion (18 duplicated pairs among 51 words) and SHAKE256-driven shuffling create uncertainty about whether a detected event corresponds to a *real* or *dummy* index. In our HQC-128 configuration, the fraction of dummy words is 18/51, so a naive association of “leakage peak $\rightarrow$ recovered index” is wrong with non-negligible probability. Beckwith’s attack maps peaks to unique per-index events in a deterministic scan; our shuffled schedule breaks that one-to-one mapping, and the duplicated dummies ensure many “hits” self-cancel in $\mathbb{F}_2$, frustrating brute-force stitching of partially recovered positions.

Claim 2 — Co-Designed Countermeasures Conceal Residual Power-Based Side-Channel Leakage. We evaluate the combined effect of all three countermeasures—parallelization, dummy insertion, and SHAKE256-driven random shuffling—using a Fixed-vs-Random TVLA. Unlike Fixed-vs-Fixed, which detects key-dependent regularity, Fixed-vs-Random assesses whether traces from a fixed key are statistically distinguishable from those obtained under random keys. An indistinguishable result implies that residual power leakage has been effectively suppressed.

As shown in Fig. 9, the full co-designed architecture reduces the fraction of significant points ($|t| > 4.5$) from **4.29%** in the baseline single-path design to only **0.09%** with 5,000 traces. While a small fraction of outliers remain, these points are randomly scattered and lack any cycle-synchronous alignment, and the number of significant samples saturates even as the trace count increases, indicating that no additional leakage emerges with more measurements.

Following the cycle-level leakage model of Beckwith *et al.* [BZKG24], the baseline reveals periodic leakage peaks every 200 samples—one per index—enabling key-dependent correlation across W_{dense} iterations. Fig. 8 zooms into a single `sparse_BRAM[i]` access. In Beckwith’s CPA on RacingBIKE [RBCGG22], the attack modeled the 32-bit HD sequence of the shift-and-add port over W_{dense} cycles to correlate each index access with power samples. In contrast, our parallel architecture removes these leakage anchors: each iteration mixes two randomized index contributions, breaking the deterministic HD pattern and leaving no stable points for correlation.

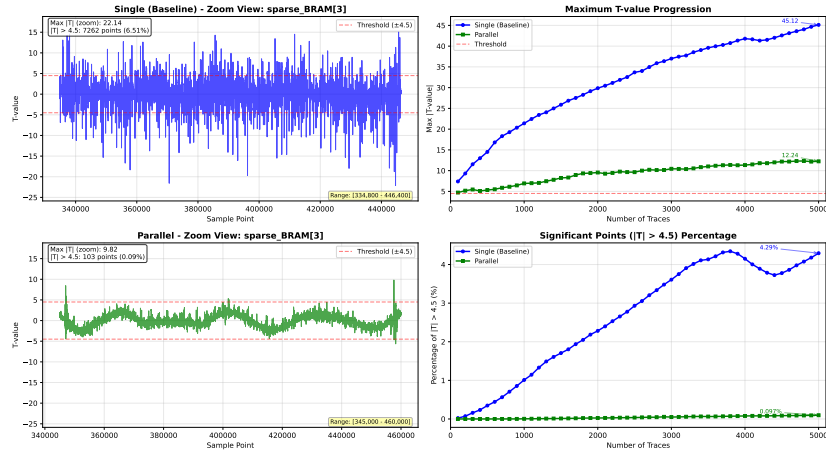


Figure 9: Fixed-vs-Random TVLA comparison between the baseline (blue) and the proposed method (green) multipliers. The left plots show representative power traces and corresponding t -values, while the right plots summarize how the maximum $|t|$ and the number of significant points ($|t| > 4.5$) evolve as the number of traces increases from 100 to 5000.

5.2 Performance

Overall results. Table 4 summarizes FPGA synthesis results of the proposed multiplier across the HQC parameter sets. Compared to prior shift-and-add designs [DXN⁺23, THKX23], our co-designed architecture achieves a $1.25\times$ **speedup** at HQC-1 (36.5K $\rightarrow$ 29.3K cycles) while maintaining near-SOTA AT efficiency. Although the slice count increases moderately due to the parallel datapath and shuffle controller, the overall AT product remains competitive because of the proportional cycle reduction. Unlike previous designs, which lack countermeasures against CPA, the proposed architecture simultaneously delivers high throughput and intrinsic resistance to power-based side-channel attacks within a single lightweight framework. For larger parameter sets (HQC-3 and HQC-5), the same architectural scaling holds, demonstrating that the two-index parallel structure and fixed-depth shift alignment adapt efficiently to higher security levels without exponential area growth.

Table 4: FPGA implementation results for HQC sparse polynomial multipliers based on the secret weight ω on Xilinx Artix-7 xc7a200t (32-bit bandwidth).

Reference	LUT	FF	Slices	BRAM (36Kb)	Cycles	Area-Time (Slices $\times$ Cycles)	CPA Prot.
HQC-1 ($\omega = 66$)							
[THKX23]	214	152	146*	2	36,565	5,338,490	$\times$
[DXN ⁺ 23]	412	189	139	1	36,698	5,101,022	$\times$
This Work	657	159	202	2	29,326	5,923,852	$\checkmark$
HQC-3 ($\omega = 100$)							
[THKX23]	238	156	172*	4	111,301	19,143,772	$\times$
[DXN ⁺ 23]	387	193	131	2	112,402	14,724,662	$\times$
This Work	648	164	200	3	98,041	19,608,200	$\checkmark$
HQC-5 ($\omega = 131$)							
[THKX23]	211	158	168*	8	236,194	39,680,592	$\times$
[DXN ⁺ 23]	397	199	134	3	236,457	31,685,238	$\times$
This Work	648	164	209	5	225,805	47,193,245	$\checkmark$

Note. *Slice count for [THKX23] estimated from reported CLB usage.

Cross-scheme applicability. To confirm that the design generalizes beyond HQC, we applied the same architecture to BIKE-1 parameters ($n=12,323$, $\omega=71$) for comparison with the RacingBIKE implementation [RBCGG22]. As shown in Table 5, our design completes each multiplication in **19.0K cycles**—a $1.46\times$ **speedup** over the sequential baseline—while maintaining comparable area (194 slices vs. 132). The resulting AT product (3.69M) remains nearly identical to the reference, indicating that the added countermeasures introduce minimal cost even under smaller BIKE parameters.

Table 5: FPGA implementation results for **BIKE-1** sparse polynomial multipliers ($n=12,323$, $\omega=71$, 32-bit bandwidth) on Xilinx Artix-7 xc7a200t FPGA.

Reference	LUT	FF	Slices	BRAM (36Kb)	Cycles	Area-Time (Slices $\times$ Cycles)	CPA Prot.
BIKE-1 ($n=12,323$, $\omega=71$)							
[RBCGG22]	319	127	132	2	27,691	3,655,212	$\times$
This Work	647	155	194	2	19,036	3,692,984	$\checkmark$

Qualitative synergy analysis. Table 6 qualitatively compares the effect of each individual countermeasure. Dummy insertion alone obscures index-dependent patterns but increases latency by $1.55\times$, while parallelization halves the cycle count at the cost of a $7.9\times$ area overhead due to deep shift alignment under single-port BRAMs. Random shuffling further desynchronizes power traces without additional hardware cost. When combined, however, the three techniques complement one another: the dummy mechanism bounds shift distances, enabling shallower parallel alignment and recovering much of the cycle loss while reducing area from $7.9\times$ to only $2.3\times$ over baseline. This **dummy-parallel synergy** achieves a $0.79\times$ cycle improvement with strong first-order side-channel resistance, validating the co-design approach described in Section 1.1.

Table 6: Qualitative effect of each countermeasure (relative to baseline shift-and-add).

Configuration	Cycles	Slices	Side-Channel Resistance
Baseline	36.8K	88	–
+ Dummy Insertion	56.9K ($\uparrow 1.55\times$ slower)	88	$\uparrow$ (real index obfuscation)
+ Parallelization	18.9K ($\downarrow 0.51\times$ faster)	697 ($\uparrow 7.9\times$ area)	$\uparrow$ (CPA hypothesis complexity)
+ Random Shuffling	–	–	$\uparrow$ (trace desynchronization)
+ Dummy + Parallel + Shuffle	29.2K ($\downarrow 0.79\times$ faster)	202 ($\uparrow 2.3\times$ area)	$\odot$ (highest SCA resistance achieved)

Legend: $\uparrow$ increase / improvement, $\downarrow$ decrease, – no significant change, $\odot$ strong improvement.
baseline: 36.8K cycles, 88 slices (Artix-7 xc7a200t, 32-bit bandwidth).

Summary. We have shown that **side-channel protection need not compromise performance**. Despite using single-port BRAMs, our multiplier achieves $1.25\times$ speedup with only $2.3\times$ area overhead while suppressing first-order leakage to near-background levels (0.09% significant points). The design principles extend naturally to other code-based KEMs like BIKE.

6 Conclusion

This paper presented a co-designed sparse-dense polynomial multiplier for HQC that integrates dummy insertion, randomized scheduling, and two-index parallel execution within a compact architecture. The proposed design resolves the long-standing trade-off between performance and side-channel resistance in conventional shift-and-add multipliers.

Implementation on a Xilinx Artix-7 FPGA achieved a $1.25\times$ speedup at the HQC-1 parameter set with **near-state-of-the-art area-time efficiency**. Test Vector Leakage Assessment (TVLA) results show that significant leakage points drop from 4.29% to 0.09%, confirming strong first-order side-channel resistance, while the parallel structure theoretically increases CPA complexity to quadratic (n^2) per iteration. Based on the proportion of polynomial multiplications in the full HQC flow, this speedup is expected to yield an overall acceleration of about $1.13\times$ for KeyGen, Encap, and Decap phases.

The bounded-distance dummy mechanism and fixed-depth shift-register alignment provide a tunable trade-off between area and latency, allowing efficient adaptation to various FPGA resource budgets. Overall, the proposed architecture demonstrates that parallelism and side-channel robustness can be synergistically co-optimized, offering a practical template for secure and efficient HQC and BIKE hardware accelerators.

Acknowledgments

This work was supported by Samsung Electronics Co., Ltd(IO250219-12010-01).

References

- [AAC⁺22] Gorjan Alagic, Daniel Apon, David Cooper, Quynh Dang, Thinh Dang, John Kelsey, Jacob Lichtinger, Yi-Kai Liu, Carl Miller, et al. Status report on the third round of the nist post-quantum cryptography standardization process. 2022.
- [ABB⁺22] Nicolas Aragon, Paulo Barreto, Slim Bettaieb, Loic Bidoux, Olivier Blazy, Jean-Christophe Deneuville, Philippe Gaborit, Santosh Ghosh, Shay Gueron, Tim Güneysu, et al. Bike: bit flipping key encapsulation. 2022.
- [ABC⁺25] Gorjan Alagic, Maxime Bros, Pierre Ciadoux, David Cooper, Quynh Dang, Thinh Dang, John Kelsey, Jacob Lichtinger, Yi-Kai Liu, Carl Miller, et al. Status report on the fourth round of the nist post-quantum cryptography standardization process, 2025.
- [ABP25] Francesco Antognazza, Alessandro Barengi, and Gerardo Pelosi. An efficient and unified rtl accelerator design for hqc-128, hqc-192, and hqc-256. *IEEE Transactions on Computers*, 2025.
- [ABPS24] Francesco Antognazza, Alessandro Barengi, Gerardo Pelosi, and Ruggero Susella. A high efficiency hardware design for the post-quantum kem hqc. In *2024 IEEE International Symposium on Hardware Oriented Security and Trust (HOST)*, pages 431–441. IEEE, 2024.
- [ALCZ20] Dorian Amiet, Lukas Leuenberger, Andreas Curiger, and Paul Zbinden. Fpga-based sphincs+ implementations: Mind the glitch. In *2020 23rd Euromicro Conference on Digital System Design (DSD)*, pages 229–237. IEEE, 2020.
- [BCL⁺17] Daniel J Bernstein, Tung Chou, Tanja Lange, Ingo von Maurich, Rafael Misoczki, Ruben Niederhagen, Edoardo Persichetti, Christiane Peters, Peter Schwabe, Nicolas Sendrier, et al. Classic mceliece: conservative code-based cryptography. *NIST submissions*, 1(1):1–25, 2017.
- [BCO04] Eric Brier, Christophe Clavier, and Francis Olivier. Correlation power analysis with a leakage model. In *Cryptographic Hardware and Embedded Systems-CHES 2004: 6th International Workshop Cambridge, MA, USA, August 11-13, 2004. Proceedings 6*, pages 16–29. Springer, 2004.
- [BDK⁺18] Joppe Bos, Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, John M Schanck, Peter Schwabe, Gregor Seiler, and Damien Stehlé. Crystals-kyber: a cca-secure module-lattice-based kem. In *2018 IEEE European Symposium on Security and Privacy (EuroS&P)*, pages 353–367. IEEE, 2018.
- [BHK⁺19] Daniel J Bernstein, Andreas Hülsing, Stefan Kölbl, Ruben Niederhagen, Joost Rijneveld, and Peter Schwabe. The sphincs+ signature framework. In *Proceedings of the 2019 ACM SIGSAC conference on computer and communications security*, pages 2129–2146, 2019.
- [BL17] Daniel J Bernstein and Tanja Lange. Post-quantum cryptography. *Nature*, 549(7671):188–194, 2017.

- [BZKG24] Luke Beckwith, Huizhen Zhou, Jens-Peter Kaps, and Kris Gaj. Power side-channel key recovery attack on a hardware implementation of bike. In *2024 Asian Hardware Oriented Security and Trust Symposium (AsianHOST)*, pages 1–6. IEEE, 2024.
- [CCJ⁺16] Lily Chen, Lily Chen, Stephen Jordan, Yi-Kai Liu, Dustin Moody, Rene Peralta, Ray A Perlner, and Daniel Smith-Tone. *Report on post-quantum cryptography*, volume 12. US Department of Commerce, National Institute of Standards and Technology . . . , 2016.
- [CPRR13] Jean-Sébastien Coron, Emmanuel Prouff, Matthieu Rivain, and Thomas Roche. Higher-order side channel security and mask refreshing. In *International Workshop on Fast Software Encryption*, pages 410–424, Berlin, Heidelberg, March 2013. Springer.
- [DHSY24] Sanjay Deshpande, James Howe, Jakub Szefer, and Dongze Yue. Sdith in hardware. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2024(2):215–251, 2024.
- [DKL⁺18] Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, Peter Schwabe, Gregor Seiler, and Damien Stehlé. Crystals-dilithium: A lattice-based digital signature scheme. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pages 238–268, 2018.
- [DXN⁺23] Sanjay Deshpande, Chuanqi Xu, Mamuri Nawan, Kashif Nawaz, and Jakub Szefer. Fast and efficient hardware implementation of hqc. In *International Conference on Selected Areas in Cryptography*, pages 297–321. Springer, 2023.
- [FHK⁺18] Pierre-Alain Fouque, Jeffrey Hoffstein, Paul Kirchner, Vadim Lyubashevsky, Thomas Pornin, Thomas Prest, Thomas Ricosset, Gregor Seiler, William Whyte, Zhenfei Zhang, et al. Falcon: Fast-fourier lattice-based compact signatures over ntru. *Submission to the NIST’s post-quantum cryptography standardization process*, 36(5):1–75, 2018.
- [GGJR⁺11] Benjamin Jun Gilbert Goodwill, Josh Jaffe, Pankaj Rohatgi, et al. A testing methodology for side-channel resistance validation. In *NIST non-invasive attack testing workshop*, volume 7, pages 115–136, 2011.
- [MAB⁺18] Carlos Aguilar Melchor, Nicolas Aragon, Slim Bettaieb, Loic Bidoux, Olivier Blazy, Jean-Christophe Deneuville, Philippe Gaborit, Edoardo Persichetti, Gilles Zémor, and IC Bourges. Hamming quasi-cyclic (hqc). *NIST PQC Round*, 2(4):13, 2018.
- [Mil86] Victor S Miller. Use of elliptic curves in cryptography. *Advances in Cryptology—CRYPTO’85 Proceedings*, pages 417–426, 1986.
- [NRR06] Svetla Nikova, Christian Rechberger, and Vincent Rijmen. Threshold implementations against side-channel attacks and glitches. In Peng Ning, Sihan Qing, and Ninghui Li, editors, *Information and Communications Security*, pages 529–545, Berlin, Heidelberg, 2006. Springer Berlin Heidelberg.
- [OC14] Colin O’flynn and Zhizhang Chen. Chipwhisperer: An open-source platform for hardware embedded security research. In *International Workshop on Constructive Side-Channel Analysis and Secure Design*, pages 243–260. Springer, 2014.

-
- [RBCGG22] Jan Richter-Brockmann, Ming-Shing Chen, Santosh Ghosh, and Tim Güneysu. Racing bike: Improved polynomial multiplication and inversion in hardware. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pages 557–588, 2022.
- [RSA78] Ronald L Rivest, Adi Shamir, and Leonard Adleman. A method for obtaining digital signatures and public-key cryptosystems. *Communications of the ACM*, 21(2):120–126, 1978.
- [Sho94] Peter W Shor. Algorithms for quantum computation: discrete logarithms and factoring. In *Proceedings 35th annual symposium on foundations of computer science*, pages 124–134. Ieee, 1994.
- [THKX23] Yazheng Tu, Pengzhou He, Çetin Kaya Koç, and Jiafeng Xie. Leap: Lightweight and efficient accelerator for sparse polynomial multiplication of hqc. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, 31(6):892–896, 2023.
- [XL21] Yufei Xing and Shuguo Li. A compact hardware implementation of cca-secure key exchange mechanism crystals-kyber on fpga. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pages 328–356, 2021.

A Comparison to First-Order Masking

We now compare our approach to first-order Boolean masking, the standard defense for arithmetic and logic datapaths on FPGAs. To be conservative, we assume a two-share masked implementation with DOM- or TI-style glitch-safe gates, regular share refresh, and duplicated memory for each share. Such designs typically incur $\approx 2.0\text{--}2.5\times$ logic overhead, $\approx 1.8\text{--}2.2\times$ cycle increase (due to refresh synchronization), and 2–3 BRAMs for per-share state [NRR06, CPRR13]. Table 7 summarizes these estimates alongside measured results for the [DXN+23] and our design.

Table 7: Resource and performance at HQC-1 (Artix-7, 32-bit bandwidth). Baseline and *This Work* are measured; masked figures are conservative estimates.

Design	Slices	BRAM	Cycles	AT (S×C)	RNG	TVLA (1st)	Notes
[DXN+23]	139	1	36,698	5.10M	–	fails	fast, vulnerable
Masked (est.)	280–350	2–3	65–80K	18–28M	yes	pass	two shares, refresh
This Work	202	2	29,326	5.92M	yes	pass (0.03%)	parallel + dummy + shuffle

Our multiplier delivers first-order TVLA non-leakage (0.03% significant samples at $|t| > 4.5$) with only $2.3\times$ area and $0.79\times$ latency relative to the baseline—a 16% AT increase ($5.10\text{M} \rightarrow 5.92\text{M}$). In contrast, generic first-order masking inflates AT by roughly $3\text{--}5\times$, making it prohibitive when slices and cycles are tightly constrained.

The security semantics differ. Masking provides information-theoretic first-order protection under standard leakage models. Our design achieves empirical first-order non-leakage (TVLA) and raises CPA complexity from $O(n)$ to $O(n^2)$ per iteration by processing two randomized indices in parallel (Claim 1). However, we do not claim provable d -th order security or resilience against advanced profiled attacks. For systems requiring higher-order guarantees, a hybrid approach—lightweight sharing on critical non-linear operations combined with our dummy/shuffle/parallel pipeline—may retain most of our performance gains while extending robustness.

On embedded FPGAs with single-port BRAM and tight area budgets, the co-designed multiplier offers a better overhead-to-security ratio than generic masking for first-order protection.

B Iterative Dummy-Insertion Algorithm

Algorithm 4 ITERATIVEDUMMYINSERT($D, \omega, d, \{T_0, \dots, T_3\}, \{W_0, \dots, W_3\}$)

Require: Array D of $\lceil \omega/2 \rceil + d$ 32-bit words, Total dummy pairs d . first $\lceil \omega/2 \rceil$ words contain sorted index pairs, remaining d words are empty. Threshold sequence $T_k = \{4096, 2048, 1024, 512\}$. the number of active words $W_k = \{33, 37, 41, 51\}$

Ensure: D updated with d inserted dummy pairs (total $\lceil \omega/2 \rceil + d$ words).

```

1:  $k \leftarrow 0$ 
2:  $W \leftarrow \lceil \omega/2 \rceil$ 
3: while  $k < 4$  do
4:   for all consecutive pairs  $(w_{2i}, w_{2i+1})$  in  $D[0:W_k-1]$  do
5:      $\delta \leftarrow w_{2i+1} - w_{2i}$ 
6:     if  $\delta > T_k$  then
7:       replace  $(w_{2i}, w_{2i+1})$  with  $(w_{2i}, w_{2i} + T_k)$ 
8:       insert dummy pair  $(w_{2i} + T_k, w_{2i+1})$  into next empty slot
9:        $W \leftarrow W + 1$ 
10:    end if
11:  end for
12:   $k \leftarrow k + 1$ 
13: end while
14: for all consecutive pairs  $(w_{2i}, w_{2i+1})$  in  $D[0:\lceil \omega/2 \rceil + d - 1]$  do
15:    $\delta \leftarrow w_{2i+1} - w_{2i}$ 
16:   replace  $(w_{2i}, w_{2i+1})$  with  $(w_{2i}, w_{2i} + \lceil \delta/2 \rceil)$ 
17:   insert dummy pair  $(w_{2i} + \lfloor \delta/2 \rfloor, w_{2i+1})$ 
18:    $W \leftarrow W + 1$ 
19:   if  $W=50$  then
20:     Break
21:   end if
22: end for
23: write all  $d$  dummy words to memory
24: return  $D$ 

```

C Auxiliary Procedures

For completeness, we summarize the auxiliary procedures used in HQC-PKE.

Reed–Muller and Reed–Solomon Codes

$\mathcal{C}.$ Encode and $\mathcal{C}.$ Decode that appear in the HQC algorithms. All binary operations are over $\mathbb{F}_2$, whereas byte symbols are identified with $\mathbb{F}_{2^8}$.

Concatenated construction. Let $\text{RS}[n_{\text{RS}}, k_{\text{RS}}]$ denote a Reed–Solomon code over $\mathbb{F}_{2^8}$, and let $\text{RM}(1, m)$ denote the first-order Reed–Muller code of length 2^m . The binary code $\mathcal{C}$ is obtained by concatenation: each RS symbol is mapped, via a fixed $\mathbb{F}_2$ -linear transform $\phi : \mathbb{F}_{2^8} \rightarrow \{0, 1\}^{2^m}$, to a valid $\text{RM}(1, m)$ codeword, whence an encoded word has length $n_{\text{RS}} \cdot 2^m$. This two-scale design delegates bit-level disturbance to the inner RM stage and residual symbol errors to the outer RS stage.

Inner code. The code $\text{RM}(1, m)$ is the evaluation set of affine Boolean functions $f(x) = a_0 \oplus a_1 x_1 \oplus \dots \oplus a_m x_m$ over all $x \in \{0, 1\}^m$. It has parameters $[2^m, m+1, 2^{m-1}]$ and thus corrects up to $t_1 = \lfloor (2^{m-1} - 1)/2 \rfloor = 2^{m-2}$ random bit errors under its standard majority-/parity-based decoder. The uniform and low-latency nature of this decoder makes RM particularly suitable as the inner component.

Outer code. Codewords of $\text{RS}[n_{\text{RS}}, k_{\text{RS}}]$ are evaluations of degree $< k_{\text{RS}}$ polynomials at n_{RS} distinct points of $\mathbb{F}_{2^8}$. Since RS is MDS, the minimum distance equals $d = n_{\text{RS}} - k_{\text{RS}} + 1$, and syndrome decoding (e.g., Berlekamp–Massey with Chien search and the Forney formula) corrects up to $t_2 = \lfloor (n_{\text{RS}} - k_{\text{RS}})/2 \rfloor$ symbol errors.

Encoding operator. Given a message m (viewed as k_{RS} bytes), the map $\mathcal{C}.$ Encode(m) first applies RS encoding to produce n_{RS} symbols and then applies ϕ symbolwise to obtain a binary word of length $n_{\text{RS}} \cdot 2^m$. In HQC this word is combined with auxiliary linear/noise terms in the ring $\mathbb{F}_2[X]/(X^n - 1)$. To reconcile the two lengths, a fixed truncation parameter $\ell = n - n_{\text{RS}}2^m$ is used, yielding the encryption update

$$v \leftarrow \mathcal{C}.$$
Encode(m) $\oplus$ Truncate($s \cdot r_2 \oplus e, \ell$).

Decoding operator. On decryption, the structured term introduced at encryption is first canceled by computing $\tilde{v} = v - \text{Truncate}(u \cdot y, \ell)$. The word $\tilde{v}$ is then partitioned into 2^m -bit blocks and each block is decoded by $\text{RM}(1, m)$, which outputs n_{RS} bytes in $\mathbb{F}_{2^8}$. The resulting RS word is finally syndrome-decoded, recovering the original k_{RS} message symbols; in short,

$$m \leftarrow \mathcal{C}.$$
Decode($v - \text{Truncate}(u \cdot y, \ell)$).

Truncation and correctness. Choosing $n > n_{\text{RS}}2^m$ preserves a convenient cyclic structure for the polynomial multiplications used elsewhere in HQC; applying the same truncation position $\ell = n - n_{\text{RS}}2^m$ in both directions ensures block alignment. Let E_{RM} be the number of blocks miscorrected by the RM stage. Decoding succeeds whenever $E_{\text{RM}} \leq t_2$. The parameters $(m, n_{\text{RS}}, k_{\text{RS}}, \ell)$ are selected so that, under the HQC noise model, the probability of $E_{\text{RM}} > t_2$ is negligible; otherwise the KEM employs implicit rejection as specified.

HQC-KEM Algorithm

HQC-KEM.Keygen. Draw a single KEM seed seed_{KEM} and expand it with an XOF to obtain a PKE seed seed_{PKE} (used to deterministically build the HQC-PKE keypair) and per-key randomness σ (used later for rejection). The KEM public key equals the PKE public key, i.e., $\text{ek}_{\text{KEM}} \equiv \text{ek}_{\text{PKE}}$. The KEM secret key packages all material needed for re-derivation and key confirmation: $\text{dk}_{\text{KEM}} = (\text{ek}_{\text{KEM}}, \text{dk}_{\text{PKE}}, \sigma, \text{seed}_{\text{KEM}})$.

Algorithm 5 HQC-KEM.Keygen()

Require: None

Ensure: Encapsulation key ek_{KEM} , decapsulation key dk_{KEM}

- ▷ Sample seed_{KEM}
 - 1: $\text{seed}_{\text{KEM}} \xleftarrow{\$} \{0, 1\}^{|\text{seed}|}$
 - ▷ Compute seed_{PKE} and randomness σ
 - 2: $\text{ct}_{\text{KEM}} \leftarrow \text{XOF.Init}(\text{seed}_{\text{KEM}})$
 - 3: $(\text{ct}_{\text{KEM}}, \text{seed}_{\text{PKE}}) \leftarrow \text{XOF.GetBytes}(\text{ct}_{\text{KEM}}, |\text{seed}|)$
 - 4: $(\text{ct}_{\text{KEM}}, \sigma) \leftarrow \text{XOF.GetBytes}(\text{ct}_{\text{KEM}}, |k|)$
 - ▷ Compute HQC-PKE keypair
 - 5: $(\text{ek}_{\text{PKE}}, \text{dk}_{\text{PKE}}) \leftarrow \text{HQC-PKE.Keygen}(\text{seed}_{\text{PKE}})$
 - ▷ Compute HQC-KEM keypair
 - 6: $\text{ek}_{\text{KEM}} \leftarrow \text{ek}_{\text{PKE}}$
 - 7: $\text{dk}_{\text{KEM}} \leftarrow (\text{ek}_{\text{KEM}}, \text{dk}_{\text{PKE}}, \sigma, \text{seed}_{\text{KEM}})$
 - 8: **return** $(\text{ek}_{\text{KEM}}, \text{dk}_{\text{KEM}})$
-

HQC-KEM.Encaps Sample an ephemeral message m and a salt, then compute $(K, \theta) \leftarrow G(H(\text{ek}_{\text{KEM}} \parallel m \parallel \text{salt}))$, so that both the session key K and the encryption coins θ are bound to the recipient's public key and salt. Encrypt m under ek_{KEM} with coins θ to obtain c_{PKE} and set the KEM ciphertext $c_{\text{KEM}} = (c_{\text{PKE}}, \text{salt})$. This makes the coins and the derived key deterministic from the same transcript.

Algorithm 6 HQC-KEM.Encaps(ek_{KEM})

Require: Encapsulation key ek_{KEM} .

Ensure: Shared secret key K , ciphertext c_{KEM} .

- ▷ Sample message m and salt
 - 1: $m \xleftarrow{\$} \{0, 1\}^{|k|}$
 - 2: $\text{salt} \xleftarrow{\$} \{0, 1\}^{|\text{salt}|}$
 - ▷ Compute shared key K and ciphertext c_{KEM}
 - 3: $(K, \theta) \leftarrow G(H(\text{ek}_{\text{KEM}} \parallel m \parallel \text{salt}))$
 - 4: $c_{\text{PKE}} \leftarrow \text{HQC-PKE.Encrypt}(\text{ek}_{\text{KEM}}, m, \theta)$
 - 5: $c_{\text{KEM}} \leftarrow (c_{\text{PKE}}, \text{salt})$
 - 6: **return** (K, c_{KEM})
-

HQC-KEM.Decaps Parse dk_{KEM} to recover $\text{ek}_{\text{KEM}}, \text{dk}_{\text{PKE}}, \sigma$, and parse $\text{c}_{\text{KEM}} = (\text{c}_{\text{PKE}}, \text{salt})$. Decrypt to m' and recompute (K', θ') . Reencrypt m' with θ' to form $\text{c}'_{\text{KEM}} = (\text{c}'_{\text{PKE}}, \text{salt})$. If decryption failed or $\text{c}'_{\text{KEM}} \neq \text{c}_{\text{KEM}}$, set the output to the rejection key $\bar{K} \leftarrow J(H(\text{ek}_{\text{KEM}}) \parallel \sigma \parallel \text{c}_{\text{KEM}})$; otherwise output K' . This implicit-rejection procedure prevents oracle leakage and ensures that only a consistent transcript yields the real shared key.

Algorithm 7 HQC-KEM.Decaps($\text{dk}_{\text{KEM}}, \text{c}_{\text{KEM}}$)

Require: Decapsulation key dk_{KEM} , ciphertext c_{KEM} .

Ensure: Shared secret key K' .

```

    ▷ Parse decapsulation key  $\text{dk}_{\text{KEM}}$ 
1:  $\text{ek}_{\text{KEM}} \leftarrow \text{dk}_{\text{KEM}}[0 : |\text{ek}_{\text{KEM}}|]$ 
2:  $\text{dk}_{\text{PKE}} \leftarrow \text{dk}_{\text{KEM}}[|\text{ek}_{\text{KEM}}| : |\text{ek}_{\text{KEM}}| + |\text{dk}_{\text{PKE}}|]$ 
3:  $\sigma \leftarrow \text{dk}_{\text{KEM}}[|\text{ek}_{\text{KEM}}| + |\text{dk}_{\text{PKE}}| : |\text{ek}_{\text{KEM}}| + |\text{dk}_{\text{PKE}}| + |\sigma|]$ 

    ▷ Parse ciphertext  $\text{c}_{\text{KEM}}$ 
4:  $\text{c}_{\text{PKE}} \leftarrow \text{c}_{\text{KEM}}[0 : |\text{c}_{\text{PKE}}|]$ 
5:  $\text{salt} \leftarrow \text{c}_{\text{KEM}}[|\text{c}_{\text{PKE}}| : |\text{c}_{\text{PKE}}| + |\text{salt}|]$ 

    ▷ Compute message  $m'$ 
6:  $m' \leftarrow \text{HQC-PKE.Decrypt}(\text{dk}_{\text{PKE}}, \text{c}_{\text{PKE}})$ 

    ▷ Compute shared key  $K'$  and ciphertext  $\text{c}'_{\text{KEM}}$ 
7:  $(K', \theta') \leftarrow G(H(\text{ek}_{\text{KEM}}) \parallel m' \parallel \text{salt})$ 
8:  $\text{c}'_{\text{PKE}} \leftarrow \text{HQC-PKE.Encrypt}(\text{ek}_{\text{KEM}}, m', \theta')$ 
9:  $\text{c}'_{\text{KEM}} \leftarrow (\text{c}'_{\text{PKE}}, \text{salt})$ 

    ▷ Compute rejection key  $\bar{K}$ 
10:  $\bar{K} \leftarrow J(H(\text{ek}_{\text{KEM}}) \parallel \sigma \parallel \text{c}_{\text{KEM}})$ 
11: if  $m' = \perp$  or  $\text{c}'_{\text{KEM}} \neq \text{c}_{\text{KEM}}$  then
12:    $K' \leftarrow \bar{K}$ 
13: endif

14: return  $K'$ 

```
